ФСП Скаут — руководство разработчика

Как устроен репозиторий, как запустить проект для разработки и как его расширять. Механики тестирования и подбора, валидация, интеграция с ФСП и перечень методов API описаны в сопроводительной документации; сценарии работы в интерфейсе — в руководстве пользователя.

1. Структура репозитория

Путь	Что внутри
`backend/app/main.py`	Точка входа FastAPI; при старте регистрирует шаблоны заданий в БД
`backend/app/api/`	Роутеры по областям: `auth`, `candidate`, `tests`, `fsp`, `employer`, `invitations`, `vacancies`, `tasks`, `admin`, `public`; `deps.py` — зависимости и проверка ролей, `serializers.py` — сборка ответов
`backend/app/core/`	`config.py` (настройки), `db.py`, `security.py`, `errors.py`
`backend/app/models.py`	Модели SQLAlchemy
`backend/app/taxonomy.py`	Справочник: специализации, грейды, навыки с синонимами
`backend/app/services/testing/`	`bank/` — шаблоны заданий, `irt.py` — модель 3PL и калибровка, `cat.py` — адаптивный тест, `service.py` — работа с БД
`backend/app/services/matching/`	`ranker.py` — ранжирование, `ltr.py` — дообучение весов, `service.py` — работа с БД
`backend/app/services/fsp/`	`client.py` — OIDC и реестр, `scoring.py` — перевод достижений в сигналы, `service.py`
`backend/app/services/`	`llm/` — провайдеры LLM, `nlp.py` — разбор текстов правилами, `resume.py` — PDF-резюме и PDF-профиль, `rubric.py` — оценка ответов на задания, `mailer.py` — письма
`backend/app/seed.py`, `seed_data.py`	Генерация демо-данных
`backend/alembic/`	Миграции схемы
`backend/tests/`	Сквозные и модульные тесты
`frontend/src/app/`	Страницы Next.js: `(auth)`, `candidate`, `employer`, `admin`
`frontend/src/components/`, `frontend/src/lib/`	Компоненты интерфейса; клиент API, сессия, типы
`fsp-mock/`	Мок ФСП ID и реестра достижений (отдельное приложение FastAPI)
`validation/`	Скрипты валидации, наборы текстов (`data/`), результаты (`out/`)
`docs/`	Исходники документации, `build.py` — сборка в HTML и PDF
`scripts/`	Локальный запуск без Docker

Правило слоёв: роутеры не содержат бизнес-логики, сервисы не зависят от HTTP. `cat.py`, `irt.py` и `ranker.py` — чистые функции без БД, поэтому скрипты валидации импортируют и

гоняют ровно тот код, что работает в продукте. Меняя эти модули, перезапускайте валидацию (раздел 6).

2. Запуск для разработки

Нужны Python 3.10+ и Node.js 20+. Без Docker используется SQLite.

Готовые скрипты: `run-local.bat` на Windows (остановка — `stop-local.bat`, логи — в `.logs`), `./scripts/dev.sh` на Linux и macOS. То же вручную, по шагам из `scripts/dev.sh`:

```
python3 -m venv .venv && . .venv/bin/activate
pip install -r backend/requirements.txt -r fsp-mock/requirements.txt
export DATABASE_URL="sqlite:///$PWD/backend/scout.db"

(cd fsp-mock && uvicorn main:app --port 8001) &          # мок ФСП ID и реестра
(cd backend && python -m app.seed --candidates 300)      # демо-данные, 1–2 минуты
(cd backend && uvicorn app.main:app --port 8000) &       # API, Swagger: http://localhost:8000/api/docs

cd frontend && npm ci
BACKEND_URL=http://127.0.0.1:8000 npm run build && BACKEND_URL=http://127.0.0.1:8000 npm start
```

Фронтенд проксирует `/api` на `BACKEND_URL`, поэтому приложение целиком доступно на http://localhost:3000.

Параметры `python -m app.seed`: `--candidates N` (по умолчанию 300), `--seed N` (по умолчанию 42), `--reset`.

3. Конфигурация

Все параметры читаются из переменных окружения или файла `.env` (`backend/app/core/config.py`). При запуске в Docker значения задаются в корневом `.env` (образец — `.env.example`), при локальном запуске — в `backend/.env`.

Переменная	По умолчанию	Назначение
`DATABASE_URL`	`postgresql+psycopg://scout:scout@localhost:5432/scout`	Строка подключения; для SQLite — `sqlite:///путь/scout.db`
`SECRET_KEY`	значение для разработки	Ключ подписи токенов; в бою — уникальный, от 32 байт
`ACCESS_TOKEN_TTL_MINUTES`	`480`	Срок жизни сессии
`COOKIE_SECURE`	`false`	`true` за HTTPS

Переменная	По умолчанию	Назначение
`EMAIL_TOKEN_TTL_HOURS`	`48`	Срок жизни ссылки подтверждения почты
`DEMO_MODE`	`true`	Письма показываются во встроенном демо-ящике; в бою — `false` и SMTP
`SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASSWORD`, `SMTP_FROM`	не задано, `587`, —, —, `noreply@scout.local`	Отправка писем
`PUBLIC_WEB_URL`, `PUBLIC_API_URL`	`http://localhost:3000`	Адрес приложения для ссылок в письмах и редиректов
`CORS_ORIGINS`	`http://localhost:3000`	Разрешённые источники
`FSP_PUBLIC_URL`	`http://localhost:8001`	Адрес ФСП ID, который видит браузер
`FSP_INTERNAL_URL`	`http://localhost:8001`	Адрес ФСП ID для бэкенда (внутри сети Docker)
`FSP_REALM`, `FSP_CLIENT_ID`, `FSP_CLIENT_SECRET`	`fsp`, `scout`, `scout-secret`	Параметры клиента OIDC
`LLM_PROVIDER`	`none`	`none` — только правила; `openai` — локальный OpenAI-совместимый сервер
`LLM_BASE_URL`, `LLM_MODEL`, `LLM_API_KEY`	`http://localhost:11434/v1`, `qwen2.5:3b`, `local`	Подключение к модели
`LLM_TIMEOUT_S`	`180`	Тайм-аут фоновых вызовов: уточнение резюме, оценка ответов

Переменная	По умолчанию	Назначение
`LLM_INTERACTIVE_TIMEOUT_S`	`45`	Тайм-аут вызовов, которые ждёт пользователь: разбор потребности
`LLM_MAX_TOKENS`, `LLM_DISABLE_THINKING`	`2048`, `false`	Параметры генерации
`GRADE_CHANGE_COOLDOWN_DAYS`	`90`	Смена грейда не чаще раза в этот срок
`SAME_GRADE_RETRY_DAYS`	`14`	Повторная попытка на тот же грейд
`TEST_MIN_ITEMS`, `TEST_MAX_ITEMS`	`8`, `25`	Длина адаптивного теста
`TEST_CONFIDENCE_Z`	`1.645`	Односторонний 95%-й интервал для решения о грейде
`PRETEST_ITEMS_PER_TEST`	`1`	Незачётных калибровочных заданий в тесте

Только для Docker (`docker-compose.yml`): `POSTGRES_PASSWORD` и `SEED_ON_START` (генерировать ли демо-данные при первом старте).

4. Как добавить шаблон задания

Шаблон — функция-генератор `gen(rng) → Instance`, зарегистрированная декоратором `@template` (`backend/app/services/testing/bank/base.py`). Каждый показ — новый экземпляр из случайного seed. Правильный ответ вычисляет сам генератор.

```
# backend/app/services/testing/bank/core.py
@template("alg_sum_range", "Сумма диапазона", DEV, {"algorithms": 1.0},
          "junior", a=1.0, b=-1.5, fmt="input", time_limit_s=60)
def _(rng):
    n = rng.randint(20, 200)
    return input_item(f"Чему равна сумма целых чисел от 1 до {n}?", n * (n + 1) // 2,
                      placeholder="число", explanation="n(n+1)/2")
```

Аргументы декоратора:

Аргумент	Смысл
`id`	Уникальный идентификатор; повтор — ошибка при импорте. Суффикс `_5opt` означает пять вариантов ответа (вероятность угадать 0,2 вместо 0,25)

Аргумент	Смысл
`title`	Название для банка заданий в админке
`specs`	Специализации, в тестах которых шаблон участвует: `backend`, `frontend`, `data_science`, `devops`, `qa`
`skills`	Навыки с весами (ключи — из `app/taxonomy.py`); по ним строятся оценки навыков
`level`	Уровень, на который рассчитано задание
`a`, `b`	Стартовая экспертная оценка дискриминативности и трудности на шкале θ (пороги грейдов: Junior −1, Middle 0, Senior 1)
`fmt`	`single` — один ответ, `multi` — несколько, `input` — ввод ответа
`time_limit_s`	Лимит времени на задание

Помощники для сборки экземпляра: `choice_item` (перемешивает варианты и убирает дубли), `multi_item`, `input_item` (принимает список допустимых ответов и числовой допуск `tol`), `near_misses` (правдоподобные неверные числа рядом с правильным).

Что важно соблюдать:

- **Изоморфность.** Все экземпляры должны решаться одним способом и быть равны по трудности: меняются числа, данные, имена и порядок вариантов, но не структура решения.
- **Ответ вычисляется.** Для заданий с кодом — выполнением или моделированием сгенерированного кода.
- **Вариативность.** `Template.variant_space()` считает различные экземпляры на выборке seed. Шаблоны с малым числом вариантов легко собрать в шпаргалку; тест допускает не больше одного такого вопроса.
- **Только `rng`.** Вся случайность берётся из переданного генератора, иначе экземпляр не воспроизведётся по seed.

Новый модуль с шаблонами нужно импортировать в `bank/__init__.py`. При старте приложения `sync_templates` добавляет новые шаблоны в таблицу `item_templates` со статусом «рабочий». Параметры `a`, `b`, `c` из кода считаются стартовыми: после калибровки в БД хранятся откалиброванные значения, и код их не перезаписывает. Шаблон, удалённый из кода, получает статус «выведен».

Проверить шаблон можно в админке: «Банк заданий» → предпросмотр экземпляра с ключом (`GET /admin/items/{tid}/preview`).

5. Схема БД и миграции

Схема описана в `backend/app/models.py`, миграции — в `backend/alembic/versions/`. В Docker контейнер бэкенда применяет миграции при старте (`alembic upgrade head`). После изменения моделей:

```
cd backend
alembic revision --autogenerate -m "описание изменения"
alembic upgrade head
```

6. Тесты и валидация

```
cd backend && pip install -r requirements-dev.txt && pytest
```

Тесты поднимают приложение на временной SQLite, отключают LLM и подключают мок ФСП напрямую, без сети (`backend/tests/conftest.py`). `test_e2e.py` проходит сквозной сценарий из ТЗ: регистрация → опрос → тест → категория → потребность → подборка → приглашение → принятие → раскрытие контактов, а также отклик на вакансию, привязку ФСП ID, ограничения доступа и кулдаун смены грейда. `test_llm_paths.py` проверяет ветки с LLM.

Валидация — симуляции с известной истиной и оценка разбора текстов:

```
cd validation
python testing_study.py      # точность, повторяемость, утечки, калибровка
python matching_study.py     # качество подбора и дообучение весов
python parsing_study.py      # разбор резюме и потребностей на наборе разработки
python llm_compare.py --rules-only   # правила на отложенном наборе текстов
python llm_compare.py                # то же с локальной моделью (llama-server на :8080)
python tune_cat.py           # подбор конфигурации адаптивного теста
```

Результаты пишутся в `validation/out/*.json`, графики — в `validation/out/*.png`. Seed зафиксированы, прогоны воспроизводимы.

7. Сборка документации

```
python docs/build.py            # Markdown → HTML → PDF в docs/build/
python docs/build.py --no-pdf   # только Markdown и HTML
```

Нужен пакет `markdown`, для PDF — `playwright` с Chromium. Скрипт подставляет в `documentation.md` на место меток `[[TOC]]`, `[[VALIDATION]]`, `[[API_TABLE]]` и `[[VERSIONS]]` оглавление, свежие числа из `validation/out`, таблицу методов из `docs/_api_table.md` и версии из `requirements.txt` и `package.json`. Поэтому раздел валидации правится в `build.py`, а не в Markdown, и после нового прогона валидации документацию нужно пересобрать.

Спецификация API лежит в `docs/openapi.json`; актуальную отдаёт работающий бэкенд: `GET /api/openapi.json`.